topic_06_custom_function

Theory, Edge Cases & Code Examples

- ✓ What is std::function?
- ✓ Type Erasure Technique
- ✓ Small Buffer Optimization (SBO)
- ✓ Signature-Agnostic Design
- ✓ Empty Function Call

→ Swipe to learn →

topic_06_custom_function

Theory

📌 1. What is std::function?

`std::function` is a **polymorphic function wrapper** that can store any callable:
- Free functions
- Member functions (with bind)
- Lambdas
- Function objects (functors)

Key characteristic: Type erasure - hides the concrete callable type behind a uniform interface.

```
std::function<int(int, int)> func;

func = [](int a, int b) { return a + b; };  // Lambda
func = std::plus<int>{};                     // Functor
func = &add_function;                         // Free function

int result = func(2, 3);  // Calls whichever is stored
```

---

topic_06_custom_function

Theory

📌 2. Type Erasure Technique (1/2)

Problem: Different callables have different types:

```
auto lambda = [](int x) { return x * 2; };  // Type: λ (compiler-generated)
struct Functor { int operator()(int x) { return x * 2; } };  // Type: Functor

// How to store both in same variable?
```

Solution: Type erasure via inheritance + templates:

```
// Base class (type-erased interface)
struct CallableBase {
    virtual int call(int) = 0;
    virtual ~CallableBase() = default;
};

// Derived template (knows concrete type)
template<typename F>
struct CallableImpl : CallableBase {
    F func_;

    CallableImpl(F f) : func_(std::move(f)) {}

    int call(int x) override {
        return func_(x);  // Invoke concrete callable
    }
};

// Wrapper
class Function {
    CallableBase* ptr_;  // Polymorphic pointer

public:
    template<typename F>
    Function(F f) : ptr_(new CallableImpl<F>(std::move(f))) {}

    int operator()(int x) {
        return ptr_->call(x);  // Virtual dispatch
    }
};
```

topic_06_custom_function

Theory

📌 2. Type Erasure Technique (2/2)

Trade-offs:

- ✅ Uniform interface
- ✅ Stores any callable
- ❌ Heap allocation
- ❌ Virtual function overhead

---

topic_06_custom_function

Theory

📌 3. Small Buffer Optimization (SBO)

To avoid heap allocation for small functors:

```
class Function {
    static constexpr size_t BUF_SIZE = 16;

    alignas(std::max_align_t) char buffer_[BUF_SIZE];
    CallableBase* ptr_;

    bool is_small(size_t size) {
        return size <= BUF_SIZE;
    }

public:
    template<typename F>
    Function(F f) {
        if (sizeof(CallableImpl<F>) <= BUF_SIZE) {
            ptr_ = new (buffer_) CallableImpl<F>(std::move(f));  // In-place
        } else {
            ptr_ = new CallableImpl<F>(std::move(f));  // Heap
        }
    }
};
```

Benefits: No allocation for small lambdas (captures ≤ 16 bytes).

---

topic_06_custom_function

Theory

📌 4. Signature-Agnostic Design (1/5)

`std::function<R(Args...)>` is a template, specialized for each signature:

```
template<typename Signature>
class Function;  // Primary template (undefined)

template<typename R, typename... Args>
class Function<R(Args...)> {  // Specialization for function signatures
    // Implementation uses Args... for call()
};
```

This allows:

```
Function<int(int, int)> add_func;
Function<void(std::string)> print_func;
```

```
#include <memory>
#include <utility>
#include <type_traits>
#include <stdexcept>

// ============================================================
// FUNCTION CLASS (Primary template)
// ============================================================

template<typename Signature>
class Function;  // Undefined

// ============================================================
// SPECIALIZATION FOR FUNCTION SIGNATURES
// ============================================================

template<typename R, typename... Args>
class Function<R(Args...)> {
private:
    // ==============================================================
    // TYPE-ERASED INTERFACE
    // ==============================================================

// ...continues on next slide
```

📌 4. Signature-Agnostic Design (2/5)

```
// ...continued from previous slide
    struct CallableBase {
        virtual R invoke(Args... args) = 0;
        virtual std::unique_ptr<CallableBase> clone() const = 0;
        virtual ~CallableBase() = default;
    };

    // ============================================================
    // CONCRETE CALLABLE WRAPPER
    // ============================================================

    template<typename F>
    struct CallableImpl : CallableBase {
        F func_;

        explicit CallableImpl(F&& f) : func_(std::forward<F>(f)) {}

        R invoke(Args... args) override {
            return func_(std::forward<Args>(args)...);
        }

        std::unique_ptr<CallableBase> clone() const override {
            return std::make_unique<CallableImpl>(func_);
        }
    };

    // ============================================================
    // MEMBERS
    // ============================================================

    std::unique_ptr<CallableBase> callable_;

public:
    // ============================================================
    // CONSTRUCTORS
    // ============================================================

    // Default constructor - empty function
    Function() noexcept : callable_(nullptr) {}

    // Nullptr constructor
// ...continues on next slide
```

📌 4. Signature-Agnostic Design (3/5)

```
// ...continued from previous slide
    Function(std::nullptr_t) noexcept : callable_(nullptr) {}

    // Callable constructor
    template<typename F,
             typename = std::enable_if_t<!std::is_same_v<std::decay_t<F>, Function>>>
    Function(F&& f)
        : callable_(std::make_unique<CallableImpl<std::decay_t<F>>>(
              std::forward<F>(f)))
    {}

    // Copy constructor
    Function(const Function& other)
        : callable_(other.callable_ ? other.callable_->clone() : nullptr)
    {}

    // Move constructor
    Function(Function&& other) noexcept
        : callable_(std::move(other.callable_))
    {}

    // ==============================================================
    // ASSIGNMENT OPERATORS
    // ==============================================================

    Function& operator=(const Function& other) {
        if (this != &other) {
            Function temp(other);
            swap(temp);
        }
        return *this;
    }

    Function& operator=(Function&& other) noexcept {
        callable_ = std::move(other.callable_);
        return *this;
    }

    Function& operator=(std::nullptr_t) noexcept {
        callable_.reset();
        return *this;
// ...continues on next slide
```

📌 4. Signature-Agnostic Design (4/5)

```
// ...continued from previous slide
    }

    template<typename F>
    Function& operator=(F&& f) {
        Function temp(std::forward<F>(f));
        swap(temp);
        return *this;
    }

    // ==============================================================
    // INVOCATION
    // ==============================================================

    R operator()(Args... args) const {
        if (!callable_) {
            throw std::bad_function_call();
        }

        return callable_->invoke(std::forward<Args>(args)...);
    }

    // ==============================================================
    // OBSERVERS
    // ==============================================================

    explicit operator bool() const noexcept {
        return callable_ != nullptr;
    }

    // ==============================================================
    // SWAP
    // ==============================================================

    void swap(Function& other) noexcept {
        std::swap(callable_, other.callable_);
    }
};

// ==============================================================
// NON-MEMBER SWAP
// ...continues on next slide
```

📌 4. Signature-Agnostic Design (5/5)

[Code continues across slides]

```
// ...continued from previous slide
// ============================================================

template<typename R, typename... Args>
void swap(Function<R(Args...)>& lhs, Function<R(Args...)>& rhs) noexcept {
    lhs.swap(rhs);
}
```

topic_06_custom_function

Edge Cases

📌 1. Empty Function Call

```
Function<void()> f;  // Empty
f();  // ← Throws std::bad_function_call
```

Check before calling:

```
if (f) {
    f();  // Safe
}
```

---

topic_06_custom_function

Edge Cases

📌 2. Return Type Mismatch

```
Function<int()> f = []() { return 42; };  // ✓ OK

Function<void()> g = []() { return 42; };  // ✗ Compile error
```

Signature must match exactly (return type + parameter types).

topic_06_custom_function

Edge Cases

📌 3. Capturing Lambda

```
int x = 10;
Function<int()> f = [x]() { return x; };  // ✓ Copies x into lambda
```

Large captures may cause heap allocation.

---

topic_06_custom_function

Edge Cases

📌 4. Self-Assignment

```
f = f;  // Must handle safely
```

Our implementation is safe (copy-and-swap idiom):

```
Function& operator=(const Function& other) {
    if (this != &other) {  // ← Self-check
        Function temp(other);
        swap(temp);
    }
    return *this;
}
```

📌 5. Storing Non-Copyable Callables

```
auto lambda = [ptr = std::make_unique<int>(42)]() {
    return *ptr;
};

Function<int()> f = std::move(lambda);  // ✓ OK (move)

Function<int()> f2 = f;  // ✗ Error: lambda not copyable
```

`std::function` requires callable to be CopyConstructible.

Alternative: Use `std::move_only_function` (C++23).

topic_06_custom_function

Code Examples

📌 1. Storing Different Callables (1/2)

This example demonstrates how std::function provides a uniform interface for storing and invoking different callable types through type erasure.

What this code does:
- Creates a single `Function<int(int, int)>` variable that can store any callable matching the signature
- Stores and invokes a free function pointer (`&add`)
- Replaces the callable with a functor object (`Multiplier`)
- Replaces it again with a lambda function
- All three different types are invoked through the same `func(a, b)` interface

Key concepts demonstrated:
- **Type erasure in action**: Different concrete types (function pointer, functor, lambda) hidden behind uniform `Function` interface
- **Polymorphic function wrapper**: Single variable seamlessly stores completely different callable implementations
- **Virtual dispatch overhead**: Each invocation goes through virtual function call to reach concrete callable
- **Assignment semantics**: Assigning new callable replaces previous one, destroying old state

Real-world applications:
- Callback systems where different modules provide different implementations
- Strategy pattern implementations where algorithms can be swapped at runtime
- Plugin architectures where behavior can be dynamically loaded
- Event handlers that may be functions, lambdas, or member function bindings

Performance implications:
- Each call incurs virtual function dispatch overhead (~5-10ns vs direct call)
- Assignment triggers heap allocation/deallocation (unless using Small Buffer Optimization)
- Type erasure prevents compiler inlining optimizations

Output:

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}
// ...continues on next slide
```

topic_06_custom_function

Code Examples

📌 1. Storing Different Callables (2/2)

[Code continues across slides]

```
// ...continued from previous slide

struct Multiplier {
    int operator()(int a, int b) const {
        return a * b;
    }
};

int main() {
    Function<int(int, int)> func;

    // Free function
    func = &add;
    std::cout << "add(2, 3) = " << func(2, 3) << '\n';  // 5

    // Functor
    func = Multiplier{};
    std::cout << "multiply(2, 3) = " << func(2, 3) << '\n';  // 6

    // Lambda
    func = [](int a, int b) { return a - b; };
    std::cout << "subtract(5, 3) = " << func(5, 3) << '\n';  // 2

    return 0;
}
```

```
['add(2, 3) = 5\nmultiply(2, 3) = 6\nsubtract(5, 3) = 2']
```

topic_06_custom_function

Code Examples

📌 2. Callback System (1/2)

This example demonstrates using std::function to implement a flexible event callback system with stateful callbacks.

What this code does:
- Implements a `Button` class that holds a callback function (`on_click_`)
- Allows external code to set custom behavior via `set_on_click()`
- The callback captures external state (`click_count`) by reference
- Each button click executes the stored callback, modifying captured state
- Demonstrates how callbacks can maintain state across multiple invocations

Key concepts demonstrated:
- **Observer pattern**: Button notifies callback without knowing its implementation
- **Stateful lambdas**: Callback captures and modifies external variable across calls
- **Decoupling**: Button class knows nothing about click_count or printing logic
- **Late binding**: Callback behavior set at runtime, not compile time
- **Null callback handling**: Check `if (on_click_)` prevents crash when no callback set

Real-world applications:
- GUI frameworks (button clicks, mouse events, keyboard handlers)
- Asynchronous I/O completion callbacks
- HTTP request handlers in web frameworks
- Game engine event systems (collision callbacks, input handlers)
- Observer pattern implementations for reactive programming

Why this matters:
- Enables inversion of control: Button doesn't dictate behavior, caller does
- Allows single Button class to support infinite behaviors without modification
- Type-safe alternative to raw function pointers that supports capturing lambdas

Output:

```
#include <iostream>
#include <vector>

class Button {
// ...continues on next slide
```

topic_06_custom_function

Code Examples

📌 2. Callback System (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    Function<void()> on_click_;

public:
    void set_on_click(Function<void()> callback) {
        on_click_ = std::move(callback);
    }

    void click() {
        if (on_click_) {
            on_click_();
        }
    }
};

int main() {
    Button btn;

    int click_count = 0;

    btn.set_on_click([&click_count]() {
        ++click_count;
        std::cout << "Button clicked! Count: " << click_count << '\n';
    });

    btn.click();
    btn.click();
    btn.click();

    return 0;
}
```

```
['Button clicked! Count: 1\nButton clicked! Count: 2\nButton clicked! Count: 3']
```

topic_06_custom_function

Code Examples

📌 3. Strategy Pattern (1/2)

This example demonstrates the Strategy design pattern using std::function to swap sorting algorithms at runtime without inheritance hierarchies.

What this code does:
- `Sorter` class wraps a comparison strategy stored in `Function<bool(int, int)>`
- Strategy is injected via constructor (dependency injection)
- Same `sort()` method works with any comparison function (ascending, descending, custom)
- Creates two `Sorter` instances with different strategies
- Demonstrates how behavior changes based on injected strategy

Key concepts demonstrated:
- **Strategy pattern without inheritance**: No need for `ComparatorBase` interface and derived classes
- **Runtime algorithm selection**: Choose sorting behavior when constructing Sorter, not at compile time
- **Composition over inheritance**: Behavior is composed through function object, not inherited
- **Uniform interface**: Client code calls `sorter.sort(vec)` regardless of comparison logic
- **Testability**: Easy to inject mock comparison functions for testing

Real-world applications:
- Sorting/filtering with configurable predicates
- Compression libraries with pluggable compression algorithms
- Database query engines with different optimization strategies
- Rendering engines with swappable shaders/pipelines
- Validation systems with configurable validation rules

Why this matters:
- Eliminates class hierarchy explosion (no need for `AscendingSorter`, `DescendingSorter` classes)
- More flexible than template-based strategy (can change at runtime)
- Enables strategy to come from configuration files, user input, or network
- Cleaner than function pointer approach (supports lambdas with captures)

Performance implications:
- Virtual dispatch overhead in comparison function (but comparison is usually dominant cost anyway)
- For performance-critical sorting, template-based approach (compile-time strategy) may be better

Output:

---

topic_06_custom_function

Code Examples

📌 3. Strategy Pattern (2/2)

```
#include <iostream>
#include <vector>
#include <algorithm>

class Sorter {
    Function<bool(int, int)> compare_;

public:
    Sorter(Function<bool(int, int)> comp) : compare_(std::move(comp)) {}

    void sort(std::vector<int>& vec) {
        std::sort(vec.begin(), vec.end(), [this](int a, int b) {
            return compare_(a, b);
        });
    }
};

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9, 2, 6};

    // Ascending
    Sorter asc([](int a, int b) { return a < b; });
    asc.sort(nums);

    std::cout << "Ascending: ";
    for (int n : nums) std::cout << n << ' ';
    std::cout << '\n';

    // Descending
    Sorter desc([](int a, int b) { return a > b; });
    desc.sort(nums);

    std::cout << "Descending: ";
    for (int n : nums) std::cout << n << ' ';
    std::cout << '\n';

    return 0;
}
```

```
['Ascending: 1 1 2 3 4 5 6 9\nDescending: 9 6 5 4 3 2 1 1']
```

What You Learned

Key Takeaways

- ✓ What is std::function?
- ✓ Type Erasure Technique
- ✓ Small Buffer Optimization (SBO)
- ✓ Signature-Agnostic Design

→ *Swipe to learn* →